# Highlights

**Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem**

Hernán Díaz Rodríguez

- Unified extreme critical path framework for neighbourhoods in the interval JSP.
- Feasibility, connectivity and no-loss proofs for any admissible interval ranking.
- LEX2 yields 5–11% narrower intervals by tightening the worst-case critical path.
- $N_2$ and $N_8$ are the strongest neighbourhoods after irace-tuned tabu search; $N_3$ worst.

# Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez[a,*]

[a]*Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain*

**Abstract**

The Job Shop Scheduling Problem with interval-valued processing times (IJSP) models real-world uncertainty where task durations are unknown but bounded. Comparing schedules then requires an interval ranking operator, and the classical notion of critical path must be extended to both extreme scenarios. I formalise *extreme critical paths* — paths critical in the best-case or worst-case scenario graph — and use them to define a neighbourhood $H(\sigma)$ for local search. I prove that $H(\sigma)$ is feasible, connected and loses no improving neighbours under any admissible interval ranking. Five neighbourhood variants ($N_1$, $N_2$, $N_3$, $N_{\text{ext}}$, $N_8$) are formulated within this framework and combined with four ranking operators (EV, LEX1, LEX2, YX) on 82 benchmark instances of up to 1000 operations. Two findings are noteworthy: *(i)* after irace-based tabu search tuning, $N_2$ and $N_8$ emerge as the two strongest neighbourhoods (statistically tied, negligible effect $r = 0.07$), both outperforming $N_{\text{ext}}$ by a medium effect and $N_1$ by a large effect; $N_3$ is weakest by a large margin. *(ii)* LEX2 yields makespan intervals 5–11% narrower than the alternatives on average; a float analysis of both extreme graphs shows LEX2 solutions concentrate critical operations in the worst-case graph, consistent with LEX2 directly tightening the worst-case critical path. *(iii)* On the full 82-instance IJSP benchmark, the irace-tuned TS-$N_2$ reduces mean RE (%) by 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances relative to the previous best published solver; the neighbourhood ranking is consistent across all instance size classes.

*Keywords:* Job shop scheduling, combinatorial optimisation, local search,

*Corresponding author

*Email address:* diazhernan@uniovi.es (Hernán Díaz Rodríguez)

neighbourhood structures, interval uncertainty, interval ranking operators

## 1. Introduction

The Job Shop Scheduling Problem (JSP) is a flagship combinatorial optimisation problem in operations research, and neighbourhood-based local search has been its most competitive algorithmic paradigm for three decades [1, 2, 3]. The effectiveness of these methods rests almost entirely on neighbourhood design: the critical-block insight of [2] — that reversing interior arcs of a critical block cannot improve the makespan — reduces neighbourhood size by two-thirds without sacrificing solution quality, and the extended neighbourhood of [3] refines this further with a heads-and-tails viability filter. In practice, however, processing times are rarely known precisely at planning time: operator variability, machine condition, and material heterogeneity introduce uncertainty that deterministic models cannot capture.

A natural and tractable representation for such uncertainty is the *closed interval* $[p^-, p^+]$, bounding the minimum and maximum possible durations [4, 5]. The resulting *Interval Job Shop Scheduling Problem (IJSP)* raises two intertwined challenges: comparing schedules requires an *interval ranking operator* [6, 7], since the makespan is now an interval; and the critical-path concept that underpins efficient neighbourhood design must be reconsidered, because different paths may become critical under different realisations of the processing times [8, 9, 10].

### *1.1. Contributions*

This paper makes the following contributions.

1. Formal definition of extreme criticality for IJSP (Section 4). I define a task or arc as *extreme critical* if it lies on a critical path of the best-case graph $G^-(\sigma)$ or the worst-case graph $G^+(\sigma)$, and prove that this definition encompasses all necessarily critical tasks and is contained within the set of possibly critical tasks, making it a computationally tractable and theoretically sound approximation of the general criticality concept of [9].
2. Neighbourhood $H(\sigma)$ based on extreme critical arcs (Section 5). I define $H(\sigma)$ as the set of all feasible processing orders reachable by reversing a single extreme critical disjunctive arc, and prove three structural

properties: feasibility (every move yields a feasible order), no loss of improving neighbours (moves outside $H(\sigma)$ cannot improve the makespan under any admissible ranking), and connectivity (every non-optimal order can reach a global optimum via a finite sequence of moves in $H(\sigma)$).

3. Five neighbourhood variants within the $H(\sigma)$ framework (Section 6). I formulate $N_1$, $N_2$, $N_3$, $N_{\text{ext}}$ and a new variant $N_8$ (boundary arc swaps extended with extra-block reinsertion moves, after [11]) as instances of $H(\sigma)$, characterising their theoretical size and the cost of generating them.
4. Empirical study on 82 benchmark instances (Section 7). Three experimental phases are reported: *(a)* a $5 \times 4$ comparison of four ranking operators at a common parameter setting (Section 7.2); *(b)* a head-to-head comparison of the five neighbourhoods after irace-based hyperparameter tuning, isolating the structural effect of the neighbourhood (Section 7.3); and *(c)* a comparison against the state of the art (Section 7.4), showing that the irace-tuned TS-$N_2$ achieves strictly better average solution quality than the previous best published solver.
5. Discovery and structural explanation of an interval-width effect (Section 7.2). One ranking operator consistently yields makespan intervals narrower than the alternatives. A float analysis of both extreme graphs reveals the structural mechanism: the operator concentrates critical operations in the worst-case graph $G^+(\sigma)$, directly tightening the worst-case critical path.

### *1.2. Paper Organisation*

Section 2 introduces the IJSP formally. Section 3 reviews related work on neighbourhood structures and published IJSP solvers. Section 4 defines extreme criticality. Section 5 defines the neighbourhood $H(\sigma)$ and proves its three structural properties. Section 6 describes the five neighbourhood variants. Section 7 reports the experimental study. Section 8 discusses the findings, and Section 9 concludes.

## 2. Background

### *2.1. The Deterministic Job Shop Problem*

The JSP consists of $n$ jobs $\{J_1, \ldots, J_n\}$ and $m$ machines $\{M_1, \ldots, M_m\}$. Each job $J_i$ is a sequence of operations $O_{i,1}, \ldots, O_{i,m}$, where operation $O_{i,j}$

must be processed on machine $M_j$ for exactly $p_{i,j}$ time units. Operations within a job must respect their precedence order, and each machine processes at most one operation at a time. The goal is to find a *feasible processing order* $\sigma$ minimising the makespan $C_{\max}$.

The *solution graph* $G(\sigma) = (V, A \cup D(\sigma))$ represents a feasible order: $V$ is the set of operations augmented with a dummy source node $S$ and sink node $T$; $A$ contains precedence arcs within jobs; and $D(\sigma)$ contains disjunctive arcs imposed by the order $\sigma$ on each machine. The makespan equals the length of the longest path in $G(\sigma)$. The classical insight of [1] is that the neighbourhood for local search should restrict to reversals of arcs on critical paths, ensuring feasibility, while [2] shows that reversals of *interior* arcs of a critical block cannot improve the makespan, allowing further pruning.

### *2.2. Interval Arithmetic and Ranking Operators*

When processing times are uncertain, I represent each as $\mathbf{p} = [p^-, p^+] \in \mathbb{IR}$, where $\mathbb{IR}$ denotes the set of closed intervals. The IJSP requires three arithmetic operations on $\mathbb{IR}$ [4]:

$$\mathbf{a} + \mathbf{b} = [a^- + b^-,\, a^+ + b^+], \tag{1}$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(a^-, b^-),\, \max(a^+, b^+)]. \tag{2}$$

Since there is no natural total order on $\mathbb{IR}$, comparing schedules requires choosing a *ranking operator*. For any interval $\mathbf{a} = [a^-, a^+]$, define its *midpoint* $m(\mathbf{a}) = \frac{a^- + a^+}{2}$ and *width* $w(\mathbf{a}) = a^+ - a^-$. I consider four rankings widely used in the literature [6, 12, 5]:

$$\mathbf{a} \leq_{\text{LEX1}} \mathbf{b} \iff a^- < b^- \vee (a^- = b^- \wedge a^+ \leq b^+) \tag{3}$$

$$\mathbf{a} \leq_{\text{LEX2}} \mathbf{b} \iff a^+ < b^+ \vee (a^+ = b^+ \wedge a^- \leq b^-) \tag{4}$$

$$\mathbf{a} \leq_{\text{YX}} \mathbf{b} \iff m(\mathbf{a}) < m(\mathbf{b}) \vee \big(m(\mathbf{a}) = m(\mathbf{b}) \wedge w(\mathbf{a}) \leq w(\mathbf{b})\big) \tag{5}$$

$$\mathbf{a} \leq_{\text{EV}} \mathbf{b} \iff m(\mathbf{a}) \leq m(\mathbf{b}) \tag{6}$$

LEX1, LEX2 and YX are *admissible* total orders on $\mathbb{IR}$ [6]: they refine the partial product order $\leq_2$ defined by $\mathbf{a} \leq_2 \mathbf{b} \iff a^- \leq b^- \wedge a^+ \leq b^+$. EV ranks intervals by their midpoint $m(\mathbf{a})$ and induces a total preorder rather than a strict total order, since two distinct intervals may share the same midpoint; it also refines $\leq_2$. This refinement property is essential for the theoretical guarantees of Section 5: any move that simultaneously decreases both $C^-_{\max}$ and $C^+_{\max}$ is an improvement under all four rankings considered here.

## 3. Related Work

*Neighbourhood structures for fuzzy and interval JSP..* The first neighbourhood for the fuzzy/interval JSP was proposed in [10], extending the van Laarhoven neighbourhood [1] to use possibly critical arcs rather than necessarily critical ones. This corresponds to what I call $N_1$ in the present framework. Subsequent work introduced $N_2$ as a filtered version of $N_1$ that retains only block-boundary arcs [13], reducing neighbourhood size by $\sim$64% with statistically identical solution quality on the fuzzy JSP. $N_3$ extends $N_2$ with three-task rearrangements at block boundaries [14]. For the related fuzzy job shop, evolutionary tabu search combining these neighbourhood structures with genetic operators has been proposed for due-date satisfaction objectives [15]. Lexicographic goal-programming strategies have also been explored for the green fuzzy FJSP with vague production targets [16] and for the green interval FJSP combining makespan and energy objectives [17]. Prior approaches to the IJSP specifically have combined neighbourhood search with genetic algorithms for makespan minimisation [18] and tardiness [19], providing early evidence that the choice of ranking operator influences solution quality.

The Nowicki–Smutnicki extended neighbourhood [3], adapted here as $N_{\text{ext}}$, was originally proposed for the crisp JSP and achieves strong empirical performance by additionally considering interior block arcs that pass a heads-and-tails viability check. Its adaptation to the IJSP is a contribution of this work, exploiting the fact that the boundary-arc result of [2] can be lifted simultaneously to both extreme graphs $G^{-}(\sigma)$ and $G^{+}(\sigma)$. A fifth variant, $N_8$, is also proposed here: it extends $N_2$ with extra-block reinsertion moves that relocate critical block endpoints to positions outside the block, adapting the idea of [11] — originally developed for the crisp JSP — to the interval setting. For the deterministic JSP and FJSP, general frameworks for feasibility checking and quality evaluation of neighbourhood moves have been proposed in [20, 21]; analogous efficiency improvements for the interval setting remain an open direction. In a complementary line, machine learning methods have been proposed to estimate quantile-based risk measures of the IJSP makespan [22].

*Metaheuristic solvers for the IJSP..* The fast-elitist Artificial Bee Colony ($ABC_{E3}$) [23] established best-known results on the 82-instance benchmark used here, combining ABC exploration with hill-climbing local search. The

Elitist-Selection ABC (*ESABC*) [24] further improved solution quality by incorporating a simulated-annealing diversity mechanism, at the cost of increased runtime. These two methods, together with the earlier GA of [18], constitute the state of the art against which the proposed tabu search is compared in Section 7.4.

## 4. Extreme Criticality for IJSP

### *4.1. Configurations and Extreme Scenario Graphs*

A *configuration* $\omega = (p_1, \ldots, p_o)$, where $o = n \times m$, assigns a deterministic duration $p_i \in \mathbf{p}_i$ to each operation $i$; let $\Omega$ denote the set of all configurations. Given a feasible processing order $\sigma$, two *extreme scenario graphs* are of particular interest:

- $G^-(\sigma)$: identical to $G(\sigma)$ but with each arc $(x, y)$ weighted by $p_x^-$ (best case).
- $G^+(\sigma)$: identical to $G(\sigma)$ but with each arc $(x, y)$ weighted by $p_x^+$ (worst case).

These correspond to the extreme configurations $\omega^+(\emptyset)$ and $\omega^+(V)$ of [9]. The makespan interval satisfies

$$\mathbf{C}_{\max}(\sigma) = [C^-_{\max}(\sigma),\, C^+_{\max}(\sigma)], \tag{7}$$

where $C^-_{\max}(\sigma)$ is the length of the longest path in $G^-(\sigma)$ and $C^+_{\max}(\sigma)$ is the length of the longest path in $G^+(\sigma)$. An illustrative example is shown in Figure 1: two jobs ($J_1$: $O_{11}(M_1, [2, 6]) \to O_{12}(M_2, [2, 3])$; $J_2$: $O_{21}(M_2, [4, 4]) \to O_{22}(M_1, [1, 4])$), with schedule $\sigma$ fixing $M_1$ order $(O_{11}, O_{22})$ and $M_2$ order $(O_{21}, O_{12})$. In $G^-(\sigma)$ the critical path runs through the $M_2$ disjunctive arc ($C^-_{\max} = 6$); in $G^+(\sigma)$ it runs through the $M_1$ disjunctive arc ($C^+_{\max} = 10$). The critical arcs in the two extreme graphs differ, motivating the need to consider both.

### *4.2. Heads, Tails and Floats*

For each extreme graph $G^*(\sigma)$ ($* \in \{-, +\}$), the *head* $r^*_x$ of task $x$ is the length of the longest path from the dummy start node to $x$, and the *tail* $q^*_x$ is the length of the longest path from the completion of $x$ to the end node. The *float* of $x$ in $G^*(\sigma)$ is

$$F^*_x(\sigma) = C^*_{\max}(\sigma) - r^*_x - p^*_x - q^*_x. \tag{8}$$

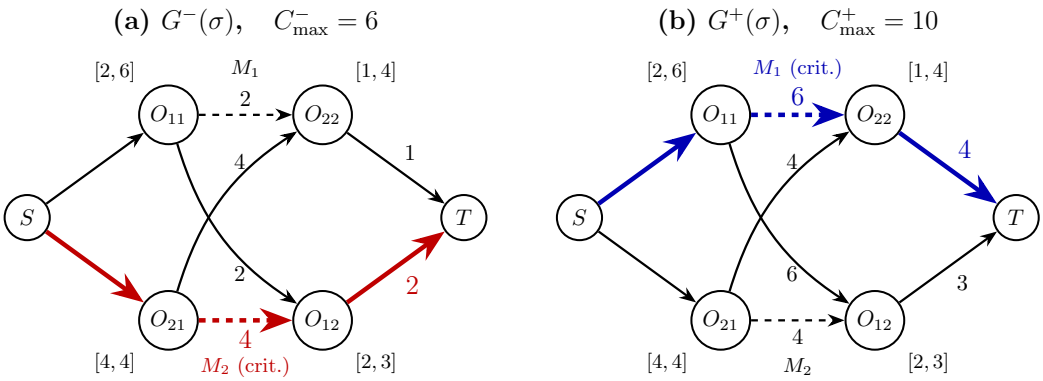



Figure 1: Illustrative example. (a) Best-case graph $G^-(\sigma)$ with its critical path highlighted in red. (b) Worst-case graph $G^+(\sigma)$ with its (different) critical path highlighted in blue. Precedence arcs are drawn solid, disjunctive arcs dashed.

A task is critical in $G^*(\sigma)$ iff $F^*_x(\sigma) = 0$. The general problem of computing the minimum float $\min_{\omega\in\Omega} F_x(\sigma, \omega)$ over all configurations — which equals zero iff $x$ is possibly critical — is NP-hard [25, 26]; efficient branch-and-bound algorithms for this harder problem are given in [27]. The extreme-scenario floats $F^-_x$ and $F^+_x$ used here are computable in linear time and provide the tractable approximation exploited in Sections 5 and 6.

### *4.3. Definition and Connection to Possibly/Necessarily Critical Tasks*

**Definition 1** (Extreme critical path)**.** *A path $P$ in $G(\sigma)$ is* extreme critical *if it is critical in $G^-(\sigma)$ or in $G^+(\sigma)$. A node, arc or block is extreme critical if it lies on an extreme critical path.*

This definition connects to the general criticality concepts of [25, 9] as follows. A task $x$ is *possibly critical* under $\sigma$ if there exists $\omega \in \Omega$ such that $F_x(\sigma, \omega) = 0$; it is *necessarily critical* if $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$ [25, 9].

**Proposition 1.** *Every extreme critical task is possibly critical. Every necessarily critical task is extreme critical.*

*Proof. (i) Extreme critical ⇒ possibly critical.* Let task $x$ be extreme critical; then $F^-_x(\sigma) = 0$ or $F^+_x(\sigma) = 0$. The extreme configurations $\omega^- = (p^-_i)_i$ and $\omega^+ = (p^+_i)_i$ both belong to $\Omega$ (since $p^\pm_i \in [p^-_i, p^+_i]$ by definition). In case $F^-_x(\sigma) = 0$: the float of $x$ under configuration $\omega^-$ equals $F_x(\sigma, \omega^-) = C^-_{\max}(\sigma) - r^-_x - p^-_x - q^-_x = F^-_x(\sigma) = 0$, so $x$ lies on a critical path for $\omega^- \in \Omega$, making it possibly critical [25, 9]. The case $F^+_x(\sigma) = 0$ is analogous with $\omega^+$.

*(ii) Necessarily critical ⇒ extreme critical.* If $x$ is necessarily critical, then $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$. Taking $\omega = \omega^- \in \Omega$ gives $F_x^-(\sigma) = 0$, so $x$ is critical in $G^-(\sigma)$, hence extreme critical. □

The practical consequence is that extreme criticality offers a computationally tractable approximation to the exact criticality predicates: computing critical paths in $G^-(\sigma)$ and $G^+(\sigma)$ each costs $O(|A \cup D(\sigma)|)$ via standard longest-path algorithms, while computing the full set of possibly critical tasks is NP-hard in general [25, 9].

## 5. The Neighbourhood $H(\sigma)$ and Its Properties

### *5.1. Definition*

**Definition 2** (Neighbourhood $H(\sigma)$)**.** *Let $\sigma_{(v)}$ denote the processing order obtained from $\sigma$ by reversing a single disjunctive arc $v \in D(\sigma)$. The neighbourhood is*

$$H(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is extreme critical}\}. \tag{9}$$

*For comparison, the corresponding neighbourhood based on* possibly *critical arcs is*

$$H'(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is possibly critical}\}. \tag{10}$$

*By Proposition 1, $H(\sigma) \subseteq H'(\sigma)$, with strict inclusion in general.*

### *5.2. No Loss of Improving Neighbours*

**Proposition 2.** *Let $\sigma \in \Sigma$ be a feasible order and let $v \in D(\sigma)$ be a disjunctive arc that is* not *extreme critical in $G(\sigma)$. Then*

$$C^-_{\max}(\sigma) \le C^-_{\max}(\sigma_{(v)}) \quad \text{and} \quad C^+_{\max}(\sigma) \le C^+_{\max}(\sigma_{(v)}), \tag{11}$$

*and consequently $\mathbf{C}_{\max}(\sigma) \le_R \mathbf{C}_{\max}(\sigma_{(v)})$ for every admissible ranking $R$.*

*Proof.* We prove $C^*_{\max}(\sigma) \le C^*_{\max}(\sigma_{(v)})$ for $* \in \{-, +\}$; both cases are identical.

Since $(x, y)$ is not extreme critical, it lies on no critical path of $G^*(\sigma)$. Let $P^*$ be any critical path of $G^*(\sigma)$, of length $C^*_{\max}(\sigma)$. $P^*$ does not contain arc $(x, y)$. Since $G^*(\sigma_{(v)})$ differs from $G^*(\sigma)$ only by replacing $(x, y)$ with $(y, x)$, every arc of $P^*$ is still present in $G^*(\sigma_{(v)})$. Hence $P^*$ is a valid directed path in $G^*(\sigma_{(v)})$ of weight $C^*_{\max}(\sigma)$, so $C^*_{\max}(\sigma_{(v)}) \ge C^*_{\max}(\sigma)$.

Applied to both extreme graphs: $C^{-}_{\max}(\sigma_{(v)}) \geq C^{-}_{\max}(\sigma)$ and $C^{+}_{\max}(\sigma_{(v)}) \geq C^{+}_{\max}(\sigma)$, i.e., $\mathbf{C}_{\max}(\sigma) \leq_2 \mathbf{C}_{\max}(\sigma_{(v)})$. Since every admissible ranking $R$ refines $\leq_2$, we obtain $\mathbf{C}_{\max}(\sigma) \leq_R \mathbf{C}_{\max}(\sigma_{(v)})$. □

**Corollary 1.** *Neighbours in $H'(\sigma) \setminus H(\sigma)$ can never improve the makespan under any of the four rankings considered.*

Corollary 1 justifies using $H(\sigma)$ instead of the larger $H'(\sigma)$: the reduction is gained at zero cost in solution quality.

*5.3. Feasibility*

**Theorem 1** (Feasibility)**.** *For every feasible $\sigma \in \Sigma$, the reversal of any extreme critical arc $v = (x, y) \in D(\sigma)$ produces a feasible processing order $\sigma_{(v)} \in \Sigma$. Hence $H(\sigma) \subseteq \Sigma$.*

*Proof.* Suppose for contradiction that $G(\sigma_{(v)})$ contains a directed cycle. Since $G(\sigma)$ is acyclic and $G(\sigma_{(v)}) = (G(\sigma) \setminus \{(x, y)\}) \cup \{(y, x)\}$, any cycle in $G(\sigma_{(v)})$ must traverse the newly added arc $(y, x)$. Therefore there exists a directed path $\pi$ from $x$ to $y$ in $G(\sigma_{(v)})$ that avoids $(y, x)$. All arcs of $\pi$ belong to $G(\sigma) \setminus \{(x, y)\}$ (the arc $(x, y)$ was removed, and $\pi$ uses none of the new arcs); so $\pi$ is a valid path from $x$ to $y$ in $G(\sigma)$ not using $(x, y)$.

In the standard JSP, $x$ and $y$ belong to the same machine but to different jobs; hence no job-precedence arc of $A$ runs directly from $x$ to $y$. Therefore $\pi$ has at least one intermediate operation $u$ distinct from $x$ and $y$. The length of $\pi$ in $G^*(\sigma)$ is

$$\ell(\pi) \;=\; p^*_x + p^*_u + \cdots \;\geq\; p^*_x + p^*_u \;>\; p^*_x,$$

since all processing times satisfy $p^*_z \geq p^-_z > 0$.

On the other hand, $(x, y)$ lies on a critical path of $G^*(\sigma)$ (denote it $CP^*(\sigma)$), which implies that the head of $y$ is tight through the arc $(x, y)$:

$$r^*_y \;=\; r^*_x + p^*_x.$$

But the path $\pi$ from $x$ to $y$ in $G(\sigma)$ yields

$$r^*_y \;\geq\; r^*_x + \ell(\pi) \;>\; r^*_x + p^*_x \;=\; r^*_y,$$

a contradiction. Hence $G(\sigma_{(v)})$ is acyclic, and $\sigma_{(v)} \in \Sigma$. □

*5.4. Connectivity*

**Theorem 2** (Connectivity)**.** *For every non-optimal $\sigma \in \Sigma$, there exists a finite sequence of moves in $H(\sigma)$ leading to a globally optimal order $\sigma^\star$.*

*Proof.* Fix any globally optimal order $\sigma^* \in \Sigma^*$ that minimises the *inversion count*

$$M(\sigma, \sigma^*) \;=\; \big|\{(x,y) \in D(\sigma) : (y,x) \in D(\sigma^*)\}\big|,$$

which counts direct machine-order disagreements between $\sigma$ and $\sigma^*$. Note that $M(\sigma, \sigma^*) = 0$ if and only if $D(\sigma) = D(\sigma^*)$, i.e., $\sigma = \sigma^*$.

*Key lemma.* If $\lambda$ is non-optimal, then there exists an extreme critical arc $(x,y) \in D(\lambda)$ with $(y,x) \in D(\sigma^*)$.

*Proof.* Since $\lambda$ is non-optimal and every ranking $R$ refines $\leq_2$, we have $\mathbf{C}_{\max}(\lambda) \not\leq_2 \mathbf{C}_{\max}(\sigma^*)$ (otherwise $\mathbf{C}_{\max}(\lambda) \leq_R \mathbf{C}_{\max}(\sigma^*)$, contradicting non-optimality). Thus either $C^-_{\max}(\lambda) > C^-_{\max}(\sigma^*)$ or $C^+_{\max}(\lambda) > C^+_{\max}(\sigma^*)$; assume without loss of generality the former. Let $P$ be a critical path in $G^-(\lambda)$, of length $C^-_{\max}(\lambda)$. If every disjunctive arc $(u,v)$ of $P$ satisfied $(u,v) \in D(\sigma^*)$, then $P$ would be a valid directed path in $G^-(\sigma^*)$ of length $C^-_{\max}(\lambda) > C^-_{\max}(\sigma^*)$, contradicting the definition of $C^-_{\max}(\sigma^*)$. Hence $P$ contains a disjunctive arc $(x,y) \in D(\lambda) \setminus D(\sigma^*)$. Since for every machine pair exactly one of $(x,y), (y,x)$ belongs to $D(\sigma^*)$, we get $(y,x) \in D(\sigma^*)$. Moreover $(x,y)$ is extreme critical because it lies on the critical path $P$ of $G^-(\lambda)$. $\square_{\text{lemma}}$

*Construction and termination.* Build the sequence $\lambda_0 = \sigma$, and at each step $k$:

(a) If $\lambda_k$ is globally optimal, stop: a globally optimal order has been reached.

(b) Otherwise, apply the key lemma to obtain an extreme critical $(x,y) \in D(\lambda_k)$ with $(y,x) \in D(\sigma^*)$. Set $\lambda_{k+1} = (\lambda_k)_{(x,y)} \in H(\lambda_k)$ (feasible by Theorem 1).

After step (b), the inversion count satisfies $M(\lambda_{k+1}, \sigma^*) = M(\lambda_k, \sigma^*) - 1$: the reversal removes $(x,y)$ from $D(\lambda_{k+1})$ (eliminating one inversion, since $(y,x) \in D(\sigma^*)$), adds $(y,x)$ (which agrees with $\sigma^*$, adding no inversion), and leaves all other arcs unchanged.

Since $M \geq 0$ and decreases by exactly one at each non-optimal step, within at most $M(\sigma, \sigma^*)$ steps the sequence must reach case (a): either a

globally optimal $\lambda_k \neq \sigma^*$, or $M(\lambda_k, \sigma^*) = 0$ implying $\lambda_k = \sigma^*$. In either case a globally optimal order is reached by a finite sequence of moves in $H$. $\square$

## 6. Neighbourhood Variants for IJSP

I instantiate five concrete neighbourhood variants within the $H(\sigma)$ framework. They differ in which subset of extreme critical arcs each one considers. Within a critical block $B = (o_1, \ldots, o_k)$, an arc $(o_i, o_{i+1})$ is a *boundary arc* if $i = 1$ or $i = k - 1$, and an *interior arc* otherwise. All five variants inherit the three structural guarantees of Section 5 (feasibility, no-loss of improving neighbours, and connectivity) for their arc-swap component: $N_2$ and $N_{\text{ext}}$ because they only exclude interior block arcs, which by Proposition 2 can never carry improving moves; $N_3$ because it is a strict superset of $N_1$; and $N_8$ because it contains all $N_2$ arc swaps (no-loss and connectivity) and its arc swaps are covered by Theorem 1 (feasibility). The fifth variant, $N_8$, additionally extends $N_2$ with *extra-block reinsertion* moves that relocate critical block endpoints to positions outside the block on the same machine; these reinsertion moves are outside the scope of Section 5 and their feasibility is verified at runtime. $N_8$ is included as an exploratory variant whose empirical behaviour is analysed in Section 7. Figure 2 illustrates a critical block and the arcs included by each variant.

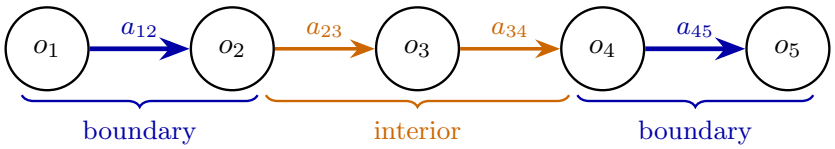
### *6.1. $N_1$: All Extreme Critical Arcs*

$N_1$ includes every disjunctive arc $(x, y)$ such that $x$ immediately precedes $y$ on the same machine in $G(\sigma)$ *and* the arc lies on a critical path of $G^-(\sigma)$ or $G^+(\sigma)$. No positional filter is applied within the block: both boundary and interior arcs of every extreme critical block are candidates. Generation cost: $O(|E|)$ per extreme graph.

### *6.2. $N_2$: Block-Boundary Filter*

$N_2$ restricts $N_1$ to arcs at the *boundary* of each extreme critical block. A block $B = \{o_1, \ldots, o_k\}$ is a maximal contiguous subsequence of operations on the same machine that all lie on a single extreme critical path. $N_2$ retains the two boundary arcs $(o_1, o_2)$ and $(o_{k-1}, o_k)$ and discards interior arcs $(o_i, o_{i+1})$, $2 \leq i \leq k - 2$. This is the direct adaptation of [2] to the interval setting: by Proposition 2 applied simultaneously to $G^-(\sigma)$ and $G^+(\sigma)$, the discarded interior arcs cannot carry improving moves under any of the four rankings, so $N_2$ inherits the same theoretical guarantees as $N_1$.

Critical block $B = (o_1, o_2, o_3, o_4, o_5)$ on machine $M_k$




| | $a_{12}$ | $a_{23}$ | $a_{34}$ | $a_{45}$ |
|---|---|---|---|---|
| $N_1$ | • | • | • | • |
| $N_2$ | • | – | – | • |
| $N_{\text{ext}}$ | • | $?_v$ | $?_v$ | • |
| $N_8$ | • | – | – | • ‡ |
| $N_3$ | • | • | • | • † |

• always included – always excluded $?_v$ conditional on `isViableSwap` ($N_{\text{ext}}$)

† $N_3$ additionally generates three rearrangement types for each boundary triple $(x, y, z)$ of consecutive extreme-critical tasks on the same machine: cyclic left-shift $(x, y, z \to y, z, x)$, cyclic right-shift $(x, y, z \to z, x, y)$, and full reversal $(x, y, z \to z, y, x)$.

‡ $N_8$ additionally generates extra-block reinsertion moves for each block endpoint (see Section 6.5).

Figure 2: Critical block $B = (o_1, \ldots, o_5)$ on machine $M_k$ and arcs considered by each neighbourhood variant.

### *6.3. $N_3$: Subsequence Reversals*

$N_3$ augments the $N_2$ boundary arc swaps with three rearrangement types applied to *boundary triples* of consecutive extreme-critical tasks. For each triple $(x, y, z)$ of successive operations on the same machine that are all extreme critical and span a block boundary (i.e. $x$ is the first task in its block or $z$ is the last), $N_3$ generates three candidate moves: a cyclic left-shift $(x, y, z \to y, z, x)$, a cyclic right-shift $(x, y, z \to z, x, y)$, and a full reversal $(x, y, z \to z, y, x)$. Each candidate is checked for feasibility before being retained. The resulting neighbourhood strictly contains $N_2$.

### *6.4. $N_{\text{ext}}$: Extended Nowicki–Smutnicki Neighbourhood*

$N_{\text{ext}}$ adapts the extended neighbourhood of [3]. It starts from $N_2$ (boundary arcs unconditionally) and additionally considers interior block arcs that pass a heads-and-tails viability check, denoted `isViableSwap`:

- Boundary arcs $(o_1, o_2)$ and $(o_{k-1}, o_k)$ are always included (as in $N_2$).
- Interior arcs $(o_i, o_{i+1})$ with $2 \leq i \leq k-2$ are included only if the `isViableSwap` check returns true.

The check `isViableSwap`$(x, y)$ computes an optimistic lower-bound estimate of the post-swap makespan by recalculating only the local heads and tails of $x$ and $y$. Because the estimate is a lower bound, discarding a swap when it exceeds the incumbent is theoretically safe.

### *6.5. $N_8$: Block-Boundary Swaps with Extra-Block Reinsertion*

$N_8$ is an exploratory variant that extends $N_2$ with *extra-block reinsertion* moves, adapting the neighbourhood of [11] to the interval setting. For a critical block $B = (o_1, \ldots, o_k)$ on a machine, with machine predecessor $o_0$ (if any) and machine successor $o_{k+1}$ (if any), $N_8$ includes all $N_2$ boundary arc swaps and additionally considers four reinsertion candidates per block:

- move $o_1$ to just after $o_k$ (between $o_k$ and $o_{k+1}$);
- move $o_k$ to just before $o_1$ (between $o_0$ and $o_1$);
- move $o_1$ to the head of the machine (before all tasks on that machine);
- move $o_k$ to the tail of the machine (after all tasks on that machine).

Each reinsertion candidate is evaluated by updating heads and tails via a localised BFS propagation; it is retained only if the resulting makespan midpoint improves on the current incumbent.

Because $N_8$ contains all $N_2$ boundary arc swaps, it satisfies all three structural guarantees of Section 5 for its arc-swap component: feasibility (Theorem 1), no-loss (Proposition 2, same argument as $N_2$), and connectivity (Theorem 2, since $N_8 \supseteq N_2$). The reinsertion moves are outside the scope of Section 5; their feasibility is verified at runtime and they carry no no-loss guarantee of their own. The motivation is the empirical question of whether extra-block moves benefit interval scheduling. As shown in Section 7.3, the answer is search-strategy dependent: $N_8$ underperforms under hill climbing (Phase A) but matches $N_2$ under irace-tuned tabu search (Phase B); Section 8.4 explains why.

## 7. Experimental Study

The experimental study is organised in three phases that address distinct questions.

**Phase A (Section 7.2).** At a *common* hyperparameter setting, I compare the four ranking operators on five neighbourhoods ($5 \times 4$ design), focusing on solution quality, the width of the returned interval, and the structural mechanism behind the LEX2–width effect.

**Phase B (Section 7.3).** After an irace-based tuning of the tabu search local search separately for each of the five neighbourhoods, I compare the five neighbourhoods on equal footing. The tuning isolates the structural effect of the neighbourhood from confounding parameter choices.

**Phase C (Section 7.4).** I compare the best Phase B configuration (TS-$N_2$) against three published IJSP metaheuristics (GA, $ABC_{E3}$, ESABC), providing external validation of the proposed approach.

### *7.1. Setup Common to All Phases*

*Instances..* The benchmark suite contains 82 instances split into two groups. The first group (*classical benchmarks*) comprises 12 instances selected from the OR-Library [28]: abz7, abz8, abz9 ($20 \times 15$); ft10 ($10 \times 10$), ft20 ($20 \times 5$); la21, la24, la25 ($15 \times 10$), la27, la29 ($20 \times 10$), la38, la40 ($15 \times 15$). The second group comprises 70 instances from the Taillard benchmark [29], covering seven size classes with 10 instances each: $15 \times 15$ (225 operations), $20 \times 15$ (300), $20 \times 20$ (400), $30 \times 15$ (450), $30 \times 20$ (600), $50 \times 15$ (750) and $50 \times 20$ (1 000). All 82 instances are derived from their deterministic counterparts by perturbing each nominal processing time $p^*_{ij}$ with a $\pm 7.5\%$ uniform symmetric interval, $\mathbf{p}_{ij} = [0.925\, p^*_{ij},\ 1.075\, p^*_{ij}]$ (15% relative width).

*Hardware and software..* All experiments were executed on an Intel Xeon E5-2680 v4 workstation (14 cores / 28 threads, 2.40 GHz, 16 GB RAM) running Windows 10 with a WSL2 Linux environment. The fEABC algorithm was implemented in C++14 and compiled with GCC using optimisation flags `-O3 -march=native -mtune=native -ffast-math`. To reduce wall-clock time, up to 24 independent runs were dispatched in parallel (one process per thread). Hyperparameter tuning (Phase B) used the irace package [30] (R 4.5.3), with a budget of 1 500 candidate evaluations per neighbourhood

and a Friedman-test racing criterion; the tuning was parallelised over 24 cores and required approximately five days of wall-clock time across the five neighbourhoods. The subsequent Phase B evaluation (12 300 runs, 15-minute time limit per run) required approximately four additional days. Statistical analysis was performed in R with paired Wilcoxon tests and Holm–Bonferroni correction. To calibrate this cluster against the hardware used in the published IJSP solvers compared in Section 7.4, I ran the Phase A fEABC+HC configuration—identical in algorithm and parameters to the one reported in [23], which used an Intel Xeon Gold 6132 at 2.60 GHz under native Linux—on the same 12 classical instances and compared average runtimes: the ratio is approximately 2× (geometric mean 2.10, range 1.4–2.8), indicating this cluster runs the identical workload roughly twice as slow. Runtime figures in Section 7.4 should be interpreted accordingly; RE (%) comparisons are hardware-independent.

*Algorithm..* All configurations use *fEABC* [23], a population-based hybrid metaheuristic that combines an Artificial Bee Colony (ABC) framework, Particle Swarm Optimisation (PSO) velocity updates for solution generation, and an embedded local search applied at each generation to a fraction of the population, with Lamarckian propagation of improvements. Solutions are encoded as permutations with repetition (*job-order*) and decoded via an insertion-based Schedule Generation Scheme (SGS); the population contains 250 individuals. The local search component differs by phase: Phase A uses first-improvement hill climbing as a neutral, operator-agnostic baseline; Phase B uses a tabu search (best-neighbour selection with a short-term tabu list) independently tuned by irace for each neighbourhood (Section 7.3; the rationale for this switch is discussed there). The stopping criterion differs by phase: Phase A terminates after 25 consecutive generations without improvement in the global best; Phase B terminates after 20 such generations or after a wall-clock limit of 900 s (15 minutes) per run, whichever is reached first. Thirty independent runs are executed for every (instance, configuration) pair.

*Hyperparameter configuration..* Phase A adopts the configuration validated in the fEABC paper [23]: JOX (Job Order Crossover), inversion mutation, food-source abandonment limit NTmax = 20 (maximum trials before a source is abandoned), elite size 40, and first-improvement hill-climbing local search applied to 75% of individuals per generation.

Phase B replaces hill climbing with tabu search and tunes six parameters independently per neighbourhood using irace [30]: crossover operator, mutation operator, elite size, NTmax, local-search target fraction, and the bad-iteration limit (maximum consecutive non-improving tabu iterations). The remaining parameters are fixed: population 250, tabu-list minimum size 3, and LEX2 ranking. LEX2 is chosen because Phase A shows it produces makespan intervals 5–11% narrower than the alternatives (Table 4) with no significant difference in midpoint quality versus EV ($p = 0.197$, $r = 0.026$, Table 3); fixing it allows the neighbourhood comparison in Phase B to be conducted under the operator that minimises solution uncertainty at no cost in makespan quality. Table 1 reports the resulting configurations.

Table 1: Irace-tuned hyperparameter configurations for Phase B. Parameters not listed are fixed: population 250, tabu-min-size 3, LEX2 ranking. *LS target*: fraction of individuals receiving local search per generation. *Bad-iter*: maximum consecutive non-improving tabu iterations before the local search terminates.

| | Crossover | Mutation | Elite | NTmax | LS target | Bad-iter |
|---|---|---|---|---|---|---|
| $N_1$ | JOX | insertion | 33 | 11 | 85.3% | 16 |
| $N_2$ | JOX | insertion | 50 | 24 | 100% | 20 |
| $N_3$ | GOX | swap | 48 | 25 | 82.8% | 30 |
| $N_{\text{ext}}$ | JOX | swap | 39 | 17 | 76.2% | 23 |
| $N_8$ | JOX | swap | 39 | 27 | 100% | 21 |

*Statistics..* Comparisons use the Friedman test (non-parametric repeated-measures ANOVA; the statistic follows a $\chi^2$ distribution with $k-1$ degrees of freedom for $k$ groups) followed by pairwise Wilcoxon signed-rank tests with Holm–Bonferroni correction. Effect size is reported as $r = |Z|/\sqrt{N}$, where $Z$ is the standardised Wilcoxon test statistic and $N$ is the number of paired observations, with the conventional thresholds (small $\geq 0.1$, medium $\geq 0.3$, large $\geq 0.5$). Each (instance, run) pair forms a paired block. The metric is the midpoint of the makespan interval, $(C^-_{\max} + C^+_{\max})/2$, which is a neutral common ground for all four operators.

### *7.2. Phase A: Operator Calibration*

Phase A is preparatory rather than central: its purpose is to identify the ranking operator that minimises makespan interval uncertainty at negligible midpoint cost, so that it can be fixed for the neighbourhood comparison in

Table 2: Phase A: mean midpoint (mid.) and median wall-clock time $t$ (s) per run, by neighbourhood and ranking operator (82 instances × 30 runs). For midpoint, lower is better.

| | EV | | LEX1 | | LEX2 | | YX | |
|---|---|---|---|---|---|---|---|---|
| Neighbourhood | mid. | $t$ | mid. | $t$ | mid. | $t$ | mid. | $t$ |
| $N_1$ | 1888.9 | 52 | 1894.6 | 33 | 1889.5 | 37 | 1887.0 | 32 |
| $N_2$ | 1888.7 | 29 | 1895.2 | 41 | 1889.2 | 33 | 1887.1 | 29 |
| $N_3$ | 1907.3 | 194 | 1914.2 | 206 | 1908.5 | 254 | 1904.9 | 201 |
| $N_8$ | 1890.5 | 30 | 1894.6 | 33 | 1891.3 | 36 | 1887.5 | 31 |
| $N_{\text{ext}}$ | 1889.0 | 33 | 1891.0 | 85 | 1895.5 | 65 | 1887.2 | 33 |

Table 3: Phase A: pairwise Wilcoxon tests for ranking operators ($n = 2460$ paired blocks, Holm–Bonferroni corrected).

| A | B | Mean A | Mean B | Diff | $p_{\text{adj}}$ | $r$ | Magnitude |
|---|---|---|---|---|---|---|---|
| LEX1 | YX | 1909.5 | 1901.5 | +8.0 | $< 10^{-3}$ | 0.509 | large |
| EV | LEX1 | 1903.3 | 1909.5 | −6.2 | $< 10^{-3}$ | 0.415 | medium |
| LEX1 | LEX2 | 1909.5 | 1903.9 | +5.6 | $< 10^{-3}$ | 0.349 | medium |
| LEX2 | YX | 1903.9 | 1901.5 | +2.4 | $< 10^{-3}$ | 0.159 | small |
| EV | YX | 1903.3 | 1901.5 | +1.8 | $< 10^{-3}$ | 0.131 | small |
| EV | LEX2 | 1903.3 | 1903.9 | −0.6 | 0.197 | 0.026 | negligible |

Phase B. The 5×4 design (five neighbourhoods, four operators, common first-improvement hill-climbing local search) gives 49,200 runs and also establishes a neutral baseline that Phase B will contrast against.

*Midpoint quality..* The Friedman test across operators yields $\chi^2(3) = 694.96$, $p < 10^{-150}$. Table 2 reports the global mean midpoint and median runtime; pairwise tests (Table 3) show that LEX1 is significantly worse than the other three (medium-to-large effects), EV and LEX2 are statistically indistinguishable ($p = 0.197$, $r = 0.026$), and YX leads on average but its advantage over EV and LEX2 carries only small effects ($r \leq 0.131$). The choice of operator has no meaningful effect on wall-clock time: Kruskal–Wallis tests return $p > 0.48$ for four of the five neighbourhoods, and the only significant case ($N_{\text{ext}}$, $p = 0.005$) involves absolute differences of $\leq 52$ s. The dominant driver of runtime is neighbourhood structure, not operator ($N_3$ is 5–8× slower than $N_2$ at the median regardless of operator).

Table 4: Mean makespan interval width by ranking operator (Phase A; mean over 5 neighbourhoods × 82 instances; one best-of-30 solution per configuration).

| Operator | Mean width | vs. EV |
|---|---|---|
| EV | 269.3 | — |
| LEX1 | 286.4 | +6.4% |
| **LEX2** | **255.0** | **−5.3%** |
| YX | 267.5 | −0.7% |

*Interval width and its mechanism..* The width $C^+_{\max} - C^-_{\max}$ measures solution uncertainty [4, 5]. Table 4 shows that LEX2 yields intervals 5.3% narrower than EV and 10.7% narrower than LEX1; the ordering LEX2 $\prec$ YX $\prec$ EV $\prec$ LEX1 holds across every neighbourhood. The effect is consistent at the instance level: LEX2 produces narrower intervals than LEX1 in 88.8%, than EV in 72.2%, and than YX in 70.5% of (instance, neighbourhood) pairs. Figure 3(a) confirms the distributional shift; panels (b) and (c) reveal its mechanism. The natural hypothesis is that LEX2 leaves more structural slack in $G^+(\sigma)$, letting the worst-case path shorten. The float analysis refutes this: LEX2 solutions have *more* critical operations and *less* per-operation float in $G^+(\sigma)$, the opposite of the slack hypothesis. The mechanism is directional: LEX2 accepts moves that reduce $C^+_{\max}$ even when $C^-_{\max}$ stays unchanged, directly tightening the upper-bound graph. This asymmetry between the two extreme graphs is LEX2's structural fingerprint; Section 8.2 provides the theoretical account.

*Conclusion and transition to Phase B..* Since EV and LEX2 are statistically equivalent on midpoint quality ($p = 0.197$) while LEX2 yields 5–11% narrower intervals, all Phase B configurations fix LEX2. Regarding neighbourhoods: under the common hill-climbing baseline the five variants produce broadly similar midpoints ($N_3$ is the exception; Table 2), with $N_{\text{ext}}$ competitive. Under irace-tuned tabu search—the subject of Phase B—$N_2$ and $N_8$ emerge as co-leaders while $N_{\text{ext}}$ falls to third; Figure 5 and Section 7.3.1 show the convergence profiles that underlie this result.

### *7.3. Phase B: Neighbourhood Comparison after Tuning*

Phase B is the central experimental contribution. Each neighbourhood is given its own irace-tuned tabu search configuration (Table 1), isolating structural neighbourhood differences from confounding parameter choices. The

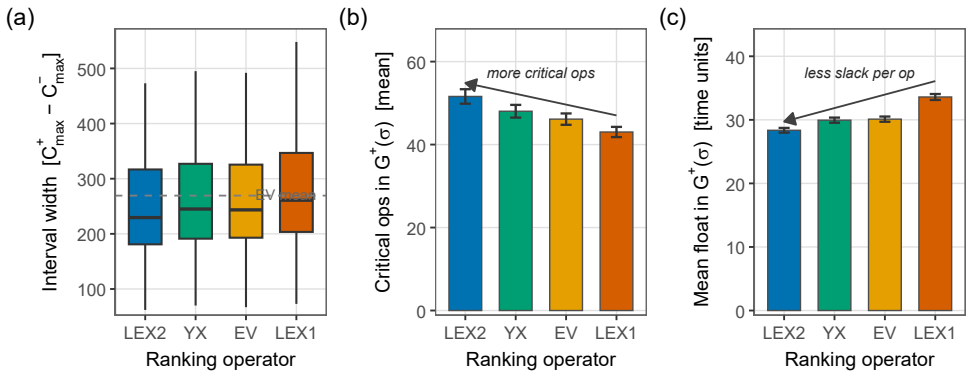



Figure 3: Phase A: operator comparison (5 neighbourhoods × 82 instances = 410 observations per operator; one best-of-30 solution per configuration). (a) Distribution of makespan interval width $C^+_{\max} - C^-_{\max}$; the dashed line marks the EV mean. (b) Mean number of critical operations in $G^+(\sigma)$ (operations with $F^+ = 0$). (c) Mean per-operation float in $G^+(\sigma)$. Panels (b) and (c) refute the slack hypothesis: LEX2 has *more* critical operations and *less* per-operation slack in $G^+(\sigma)$, confirming directional tightening of the worst-case critical path.

switch from hill climbing to tabu search is necessary: hill climbing converges quickly to a local optimum and then stalls, suppressing any structural difference between neighbourhoods; tabu search escapes local optima throughout the full time budget, allowing each neighbourhood's block geometry and candidate-set size to influence the search trajectory. Using the irace-tuned configurations, I evaluate the five neighbourhoods on the 82 benchmark instances (30 runs per instance, $5 \times 82 \times 30 = 12{,}300$ observations); the most notable configuration difference is that irace selected GOX crossover for $N_3$—the only variant using subsequence reversals—while all others retained JOX, and $N_3$ received the highest bad-iteration budget (30), consistent with its larger candidate set.

Figure 4(a) shows the per-instance relative error RE (%):

$$\mathrm{RE}\,(\%) = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \tag{12}$$

where LB is the published lower bound from the literature [29, 28, 23]. This single metric is used uniformly throughout Sections 7.3–7.4. The Friedman test yields $\chi^2(4) = 4120.58$, $p < 10^{-150}$; mean ranks are $N_2$: 2.143, $N_8$: 2.245, $N_{\mathrm{ext}}$: 2.638, $N_1$: 3.419, $N_3$: 4.555. Pairwise tests (Table 5) reveal that $N_2$ and $N_8$ are essentially tied (negligible effect, $r = 0.07$, $p = 0.001$), while both

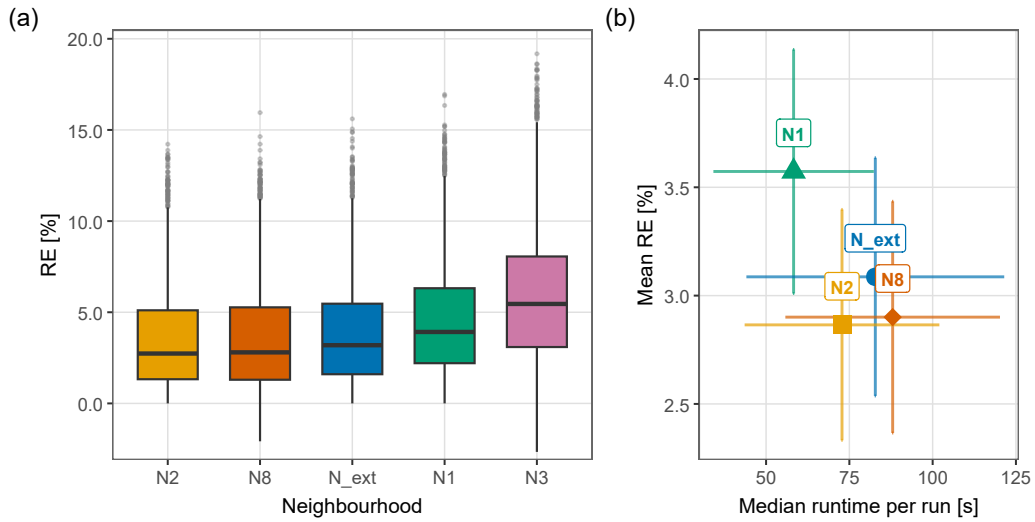



Figure 4: Phase B (irace-tuned tabu search). **(a)** Per-instance RE (%) on the makespan midpoint by neighbourhood (Eq. 12). Each box pools $82 \times 30 = 2{,}460$ observations; pairwise significance and effect sizes are reported in Table 5. **(b)** Quality–runtime trade-off (N3 omitted for scale clarity): each point is the mean RE (%) vs. mean median runtime aggregated over all instance groups; error bars are $\pm 1$ SE. Aggregated over all groups, $N_2$ achieves the lowest mean RE (%) at moderate runtime ($\approx$73 s); $N_8$ is slightly slower ($\approx$88 s) with nearly identical RE (%); $N_1$ is fastest ($\approx$58 s) but incurs a large-effect quality penalty; $N_{\text{ext}}$ sits between $N_8$ and $N_1$ on both axes.

are separated from $N_{\text{ext}}$ by a medium effect, from $N_1$ by a large effect, and from $N_3$ by a very large margin.

The picture that emerges differs markedly from the common hyperparameter setting of Phase A: $N_2$ and $N_8$—not $N_{\text{ext}}$—are the two strongest neighbourhoods after irace-based tabu search tuning. The difference between $N_2$ and $N_8$ is negligible ($r = 0.07$; indeed the only pair in Table 5 without a large or medium effect), while both sit a medium effect ahead of $N_{\text{ext}}$, a large effect ahead of $N_1$, and a very large margin above $N_3$. The rise of $N_8$ to the top tier is noteworthy: under hill climbing (Phase A), its extra-block reinsertion moves were outweighed by their cost and the difficulty the midpoint-based filter had in selecting interval-improving candidates; under tabu search, the tabu mechanism prevents cycling through the enlarged candidate set, allowing the reinsertion moves to contribute genuine diversity to the search trajectory.

Decomposing by instance group (Table 6), $N_2$ posts the lowest group-

Table 5: Phase B: pairwise Wilcoxon tests for tuned neighbourhoods ($n = 2460$ paired blocks, Holm–Bonferroni corrected). All differences are statistically significant. † p-value underflows to zero in double precision; effect size exceeds 0.68 (large).

| A | B | Mean A | Mean B | Diff | $p_{\text{adj}}$ | $r$ | Magnitude |
|---|---|---|---|---|---|---|---|
| $N_2$ | $N_3$ | 1846.5 | 1895.5 | −49.0 | $< 10^{-3}$ | −† | large† |
| $N_3$ | $N_8$ | 1895.5 | 1847.8 | +47.7 | $< 10^{-3}$ | −† | large† |
| $N_3$ | $N_{\text{ext}}$ | 1895.5 | 1853.8 | +41.7 | $< 10^{-3}$ | −† | large† |
| $N_1$ | $N_3$ | 1868.6 | 1895.5 | −26.9 | $< 10^{-249}$ | 0.682 | large |
| $N_1$ | $N_2$ | 1868.6 | 1846.5 | +22.1 | $< 10^{-222}$ | 0.643 | large |
| $N_1$ | $N_8$ | 1868.6 | 1847.8 | +20.9 | $< 10^{-196}$ | 0.603 | large |
| $N_1$ | $N_{\text{ext}}$ | 1868.6 | 1853.8 | +14.9 | $< 10^{-122}$ | 0.476 | medium |
| $N_{\text{ext}}$ | $N_2$ | 1853.8 | 1846.5 | +7.3 | $< 10^{-61}$ | 0.337 | medium |
| $N_{\text{ext}}$ | $N_8$ | 1853.8 | 1847.8 | +6.0 | $< 10^{-38}$ | 0.267 | small |
| $N_2$ | $N_8$ | 1846.5 | 1847.8 | −1.3 | 0.001 | 0.070 | negligible |

level RE (%) on 11 of 19 groups (sole winner on 9, tied on 2); $N_8$ leads on 9 groups (sole on 7, tied on 2). $N_{\text{ext}}$ achieves the minimum RE (%) on only LA38 (1.88% vs $N_2$'s 1.91%, a margin of 0.03 pp). The runtime columns tell a different story from Phase A: $N_1$ is the fastest neighbourhood on small and medium instances (tai30×20: 102 s vs $N_2$'s 170 s) but trails on quality by a large effect; $N_{\text{ext}}$ is competitive on small instances (LA21: 8 s) but becomes the *slowest* viable neighbourhood on the largest instances (tai50×15: 354 s; tai50×20: 695 s), exceeding $N_2$'s runtimes (181 s; 554 s) by a factor of ≈2 at inferior quality. $N_3$ again occupies the worst position on both dimensions: its median runtime reaches 898 s at tai50×20—effectively hitting the 900 s time limit—with the worst quality on every instance group.

Figure 4(b) visualises the quality–runtime trade-off aggregated across instance groups: $N_2$ achieves the lowest mean RE (%) at moderate runtime, with $N_8$ nearly indistinguishable in quality at slightly higher cost; $N_1$ is the fastest but sits at substantially worse quality (a large effect separates it from $N_2$). Considered jointly, $N_2$ offers the best efficiency profile across all instance sizes: it ties with $N_8$ for the best quality and is the faster of the two for small and medium instances, while being faster than $N_{\text{ext}}$ on all instances with $\geq 450$ operations. $N_8$ is equally recommended when the instance is large and runtimes exceeding 500 s are acceptable, particularly for the tai50×15 class where it achieves the lowest RE (%) (0.22% vs $N_2$'s 0.32%). The practical implication is that $N_2$ is the default choice for most settings, and $N_8$

Table 6: Phase B: mean RE (%) and median runtime $t$ (s) per instance group and neighbourhood (irace-tuned TS, 30 runs per instance). Bold: minimum RE (%) in each row. Groups are ordered as in Table 7 and the supplementary material [31].

| | $N_2$ | | $N_8$ | | $N_{\text{ext}}$ | | $N_1$ | | $N_3$ | |
|---|---|---|---|---|---|---|---|---|---|---|
| Group | RE | $t$ | RE | $t$ | RE | $t$ | RE | $t$ | RE | $t$ |
| ABZ7 | **3.03** | 33 | 3.09 | 44 | 3.24 | 30 | 3.52 | 23 | 4.07 | 262 |
| ABZ8 | 6.64 | 40 | **6.21** | 57 | 6.51 | 31 | 6.91 | 25 | 7.97 | 210 |
| ABZ9 | **6.31** | 41 | 6.32 | 48 | 6.78 | 32 | 6.82 | 24 | 8.50 | 446 |
| FT10 | 0.99 | 7 | **0.96** | 10 | 0.97 | 5 | 1.15 | 5 | 2.02 | 111 |
| FT20 | 0.23 | 9 | **0.18** | 12 | 0.29 | 11 | 0.79 | 9 | 1.45 | 73 |
| LA21 | 1.39 | 12 | **1.37** | 16 | 1.48 | 8 | 1.95 | 8 | 3.03 | 111 |
| LA24 | 1.37 | 11 | 1.52 | 16 | **1.36** | 8 | 1.49 | 7 | 2.24 | 102 |
| LA25 | 1.31 | 11 | **1.28** | 16 | 1.56 | 9 | 1.42 | 8 | 2.52 | 101 |
| LA27 | **1.69** | 20 | 1.75 | 26 | 1.91 | 20 | 2.23 | 15 | 2.89 | 229 |
| LA29 | **2.87** | 23 | 3.20 | 32 | 3.15 | 22 | 3.43 | 18 | 6.31 | 317 |
| LA38 | 1.91 | 20 | 1.94 | 28 | **1.88** | 15 | 3.10 | 13 | 3.56 | 338 |
| LA40 | 1.09 | 18 | **1.08** | 24 | 1.14 | 13 | 1.51 | 12 | 1.60 | 340 |
| 15×15 | **2.04** | 20 | 2.05 | 27 | 2.24 | 15 | 2.38 | 12 | 3.05 | 513 |
| 20×15 | **3.58** | 43 | 3.66 | 55 | 3.85 | 33 | 4.28 | 26 | 5.50 | 703 |
| 20×20 | **5.34** | 62 | 5.52 | 77 | 5.69 | 45 | 5.86 | 35 | 6.96 | 834 |
| 30×15 | **3.68** | 108 | 3.88 | 138 | 4.11 | 98 | 5.00 | 76 | 7.12 | 874 |
| 30×20 | **8.62** | 170 | 8.78 | 225 | 9.07 | 129 | 9.95 | 102 | 12.08 | 593 |
| 50×15 | 0.32 | 181 | **0.22** | 218 | 0.61 | 354 | 1.48 | 298 | 2.50 | 812 |
| 50×20 | **2.03** | 554 | 2.11 | 603 | 2.82 | 695 | 4.63 | 391 | 6.75 | 898 |
| *Grand mean* | 3.47 | 73 | 3.55 | 88 | 3.83 | 83 | 4.51 | 58 | 5.93 | 414 |

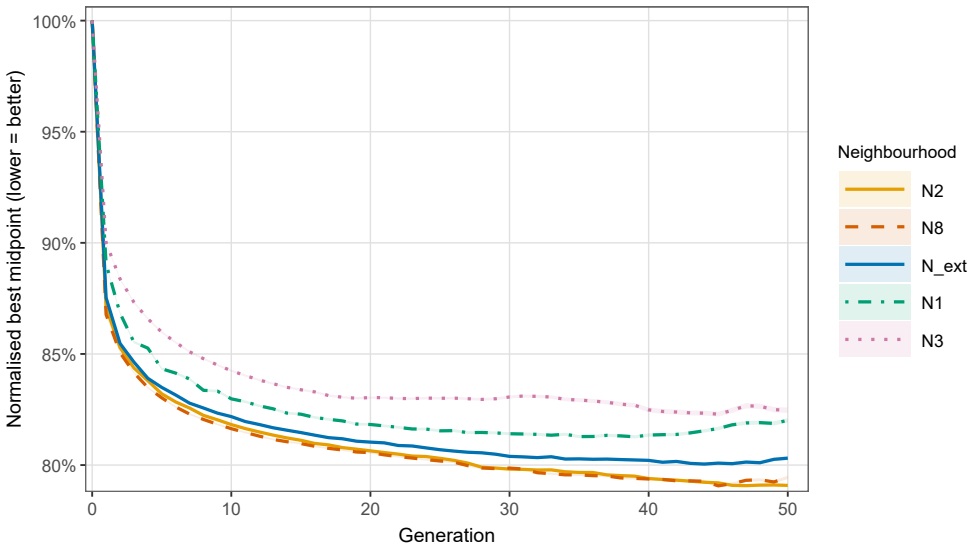



Figure 5: Per-generation normalised best-midpoint profiles, by neighbourhood (Phase B, irace-tuned tabu search; mean over 82 instances $\times$ 30 runs per neighbourhood). The shaded ribbons represent $\pm 1$ standard error. All five neighbourhoods converge quickly (within $\sim$10 generations) and then plateau; $N_2$ and $N_8$ reach the lowest final values (0.793 and 0.799), consistent with their joint statistical lead in Table 5.

the preferred alternative for large instances with a generous time budget.

*7.3.1. Convergence Profiles under Tabu Search*

Figure 5 shows the per-generation convergence profiles of the five irace-tuned tabu search configurations, averaged over all 82 instances and 30 runs. All five neighbourhoods converge rapidly in the first 5–10 generations, after which the curves separate and plateau. $N_2$ and $N_8$ reach the lowest normalised plateaus (0.793 and 0.799 respectively), consistent with their joint statistical lead in Table 5. $N_{\text{ext}}$ converges to a slightly higher plateau (0.808), and $N_1$ and $N_3$ trail at 0.816 and 0.834. Notably, $N_8$ plateaus slightly faster than $N_2$ and $N_{\text{ext}}$, reflecting its larger per-iteration candidate set: fewer tabu iterations are needed to exhaust improvement opportunities. The convergence profiles provide a generation-level counterpart to the statistical results of Table 5: the neighbourhood ranking is already visible from generation 10 onward and stable thereafter.

Table 7: Comparison with published IJSP solvers on 12 classical instances (Phase B, irace-tuned TS, $N_2$). Metric: RE (%) $= (E[\mathbf{C}_{\max}] - \mathrm{LB}) / \mathrm{LB} \times 100$ (Eq. 12). LBs from [23]. GA from [18]; $ABC_{E3}$ from [23]; *ESABC* from [24]. All methods: 30 independent runs. Bold: row-minimum average RE. $t$ = median runtime (s).

| | | *GA* [18] | | | $ABC_{E3}$ [23] | | | *ESABC* [24] | | | *TS*-$N_2$ (ours) | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Instance | LB | Best | Avg | $t$ | Best | Avg | $t$ | Best | Avg | $t$ | Best | Avg | $t$ |
| ABZ7 | 656 | 6.33 | 12.50 | 1.8 | 5.26 | 7.32 | 4.5 | 4.19 | 6.73 | 6.5 | 1.83 | **3.03** | 32 |
| ABZ8 | 645 | 11.32 | 18.48 | 1.8 | 8.99 | 12.06 | 4.2 | 8.68 | 10.95 | 7.7 | 4.50 | **6.64** | 39 |
| ABZ9 | 661 | 13.01 | 17.97 | 2.2 | 9.68 | 13.13 | 6.0 | 8.55 | 11.19 | 8.7 | 5.07 | **6.31** | 40 |
| FT10 | 930 | 1.83 | 5.23 | 0.5 | 1.08 | 4.11 | 1.6 | 0.59 | 3.01 | 1.8 | 0.59 | **0.99** | 7 |
| FT20 | 1165 | 1.46 | 4.35 | 0.7 | 0.69 | 1.73 | 2.7 | 0.69 | 1.78 | 2.9 | 0.09 | **0.23** | 9 |
| la21 | 1046 | 3.15 | 5.01 | 1.1 | 2.58 | 5.01 | 1.8 | 2.06 | 3.96 | 2.9 | 0.57 | **1.39** | 11 |
| la24 | 935 | 4.06 | 6.34 | 0.8 | 2.25 | 5.06 | 2.7 | 3.53 | 4.95 | 2.6 | 0.80 | **1.37** | 10 |
| la25 | 977 | 1.94 | 5.11 | 1.0 | 1.94 | 3.88 | 2.4 | 1.33 | 2.74 | 3.0 | 0.20 | **1.31** | 10 |
| la27 | 1235 | 4.57 | 10.22 | 1.3 | 2.75 | 4.66 | 4.1 | 2.75 | 4.12 | 5.8 | 0.97 | **1.69** | 19 |
| la29 | 1152 | 11.11 | 14.23 | 1.1 | 5.51 | 8.65 | 4.4 | 4.99 | 7.03 | 6.7 | 1.95 | **2.87** | 21 |
| la38 | 1196 | 6.02 | 9.16 | 1.4 | 4.52 | 6.88 | 6.0 | 3.01 | 5.83 | 7.2 | 0.71 | **1.91** | 20 |
| la40 | 1222 | 5.07 | 8.74 | 1.2 | 1.88 | 4.21 | 3.0 | 2.74 | 4.11 | 3.7 | 0.70 | **1.09** | 17 |
| *Mean* | | | 9.78 | | | 6.39 | | | 5.53 | | | **2.40** | |

### *7.4. Comparison with Published IJSP Solvers*

I compare the irace-tuned TS under neighbourhood $N_2$ (Phase B best, Section 7.3) against three published IJSP metaheuristics on the same benchmark instances: a Genetic Algorithm (GA) [18], a fast-elitist Artificial Bee Colony ($ABC_{E3}$) [23], and an Elitist-Selection ABC (*ESABC*) [24]. All methods use 30 independent runs per instance; results for the published solvers are taken directly from the cited papers. The comparison metric is RE (%) (Equation 12), where the LB values are from [23] for the classical instances and from [29] for the Taillard instances.

*Classical instances..* Table 7 compares all four solvers on the 12 classical instances (ABZ7–9, FT10, FT20, LA21/24/25/27/29/38/40). TS-$N_2$ achieves a mean RE of **2.40**%, against 5.53% for ESABC, 6.39% for $ABC_{E3}$, and 9.78% for GA, a reduction of 56.6% relative to the previous best (ESABC). TS-$N_2$ attains the lowest mean RE on all 12 instances individually; the largest absolute gains arise on FT20 (0.23% vs. 1.78% for ESABC) and la27 (1.69% vs. 4.12%).

*Taillard instances..* Per-instance results on all 70 Taillard instances (TA1–TA70, seven size classes from 15×15 to 50×20) are provided as supplementary material [31]. GA results are unavailable for the Taillard IJSP instances; the comparison covers $ABC_{E3}$ and ESABC only; TS-$N_8$ is included alongside TS-$N_2$ for reference. TS-$N_2$ achieves a grand mean RE of **3.66**% across all 70 instances, against 8.66% for ESABC and 9.39% for $ABC_{E3}$, a 57.7% reduction relative to ESABC. TS-$N_2$ achieves the strictly lower average RE than both ABC methods on every one of the 70 instances. The improvement is consistent across size classes and most pronounced on tai50×15 (TA51–TA60; group mean: TS-$N_2$ = 0.32% vs. ESABC = 4.55%), where the interval TS nearly matches the lower bound—TA51 achieves RE = 0.02% and TA53 achieves RE = 0.04%. The hardest class, tai30×20 (TA41–TA50), yields a group mean of 8.62% for TS-$N_2$ versus 15.32% for ESABC; these instances remain the most challenging for all methods. TS-$N_8$ finishes marginally behind TS-$N_2$ in grand mean (3.74% vs. 3.66%), consistent with the Phase B ranking (Section 7.3), though it outperforms $N_2$ on individual instances where a single dominant critical path prevails (Section 8.4).

## 8. Discussion

### *8.1. Practical Recommendations*

The experimental results lead to a small set of actionable recommendations:

1. Choose between $N_2$ and $N_8$ based on instance size and time budget. $N_2$ is the default recommendation: it ties with $N_8$ for the best quality overall (negligible difference, $r = 0.07$) and is faster for small and medium instances (Table 6). On the tai50×15 class, $N_8$ achieves the strictly lower group mean RE (%) (0.22% vs $N_2$'s 0.32%) at a moderate extra runtime (218 s vs 181 s), consistent with its structural advantage in the single-dominant-critical-path regime [11]; for tai50×20, $N_2$ recovers the edge (2.03% vs $N_8$'s 2.11%, 554 s vs 603 s). $N_8$ is thus the preferred choice when quality is paramount on the largest 50-job instances with 15 machines, while $N_2$ is more robust across the full range.
2. Do not assume $N_{\text{ext}}$ is best on the basis of Phase A results. Under irace-tuned tabu search, $N_{\text{ext}}$ finishes third—a medium effect behind both $N_2$ and $N_8$—and becomes the *slowest* viable neighbourhood on large instances (tai50×20: 695 s vs $N_2$'s 554 s and $N_8$'s 603 s, at a 0.79 pp

quality cost; Table 6). The heads-and-tails viability filter that makes $N_{\text{ext}}$ efficient under hill climbing becomes a runtime bottleneck under tabu search with a full neighbourhood evaluation per iteration.

3. Use LEX2 if interval width is the goal; operator choice has limited impact on midpoint. LEX2 yields makespan intervals 5–11% narrower than the alternatives on average, with structural evidence for this effect (Section 8.2), and this advantage is consistent across prior work that includes per-operator hyperparameter tuning [24, 32].
4. Avoid $N_3$ in either setting. The 3-task block rearrangements it adds beyond $N_2$ are dominated on both dimensions: it is the weakest neighbourhood on solution quality and the most expensive computationally, reaching the 900 s time limit for the largest instances.

### *8.2. Theoretical Connection: Why LEX2 Narrows Intervals*

Proposition 2 guarantees that reversing a non-extreme-critical arc cannot improve either bound. LEX2 specifically targets $C^+_{\max}$, accepting moves that reduce $C^+_{\max}$ even when $C^-_{\max}$ stays put. Over many local search steps, this creates a bias toward configurations where $G^+(\sigma)$ has a shorter critical path — and, by the structure of interval arithmetic, a tighter bound on the overall interval width. The float analysis of Section 7.2 provides direct empirical confirmation: LEX2 solutions have more critical operations in $G^+(\sigma)$ and less per-operation slack, while $G^-(\sigma)$ becomes relatively looser. The asymmetry between the two extreme graphs is a structural fingerprint of the LEX2 ranking rule.

### *8.3. Why $N_3$ Underperforms*

$N_3$ is counter-intuitive: it is a strict superset of $N_2$ in the move-generation sense (boundary arc swaps plus three types of 3-task block rearrangements—cyclic left-shift, cyclic right-shift, and full reversal of boundary triples), yet it produces worse solutions. Each 3-task rearrangement candidate requires a full feasibility evaluation to detect cycles before it can be retained, which significantly increases the per-iteration cost. More importantly, these extra candidates do not carry improving moves in practice: the boundary arc swaps already capture the structurally meaningful transitions (as justified by Proposition 2), and the 3-task rearrangements add volume to the candidate pool without adding quality. Under best-improvement selection, a larger pool diluted with poor-quality candidates reduces the effective yield per evaluation, resulting in slower convergence to a worse plateau.

*8.4. Why $N_8$ Performs Well Under Tabu Search*

$N_8$ extends $N_2$ with extra-block reinsertion moves filtered by a midpoint lower-bound estimate. Under hill climbing (Phase A), this pairing is counterproductive in the interval setting: the midpoint filter discards moves that reduce $C^+_{\max}$ without improving $C^-_{\max}$—precisely those that exploit the disagreement between $G^-(\sigma)$ and $G^+(\sigma)$—so hill climbing spends budget evaluating reinsertion candidates that rarely improve the interval criterion. The result is slower convergence at lower yield compared with $N_2$, as visible in Figure 5.

Under tabu search (Phase B), this penalty reverses. Three mechanisms interact. First, the tabu list prevents revisiting recently-moved operations, forcing the search to explore the full breadth of $N_8$'s candidate set—including the reinsertion moves—rather than cycling near a local optimum where only arc swaps are informative. Second, tabu search employs best-improvement selection: among all candidate moves at each iteration (arc swaps and reinsertions alike), the globally best is chosen; in a larger neighbourhood this captures improvements that first-improvement hill climbing would miss. Third, the aspiration criterion allows a tabu move to be accepted if it improves the current best solution; this creates a channel through which reinsertion moves that improve the interval criterion in aggregate (even at a mid-iteration midpoint cost) are eventually accepted.

In the deterministic setting, the mechanism is simpler: with a single critical path, a reinsertion that improves the midpoint also improves the makespan, so the filter is calibrated and extra-block moves help under both hill climbing and tabu search [11]. The practical implication is that $N_8$ is a neighbourhood whose benefit is conditional on the search strategy: it requires a memory-based mechanism to exploit its enlarged candidate set effectively.

*8.5. Limitations and Future Work*

- Larger instances and per-operator tuning. Extending to tai100×20 (2000 operations) would require excessively long runtimes on the current cluster, but would be valuable: larger search spaces tend to sharpen structural differences between neighbourhoods, and the question of whether operator differences persist under per-operator tuning remains open beyond the 12 small instances studied in [24, 32].

- Other uncertainty levels. The 15% uniform interval width is a single noise level; sensitivity to other noise widths is left open.

- Hybrid algorithms. (LEX2, $N_{\text{ext}}$) is the natural starting point for an uncertainty-aware metaheuristic targeting interval width; (LEX2, $N_2$) offers a faster alternative with a small quality trade-off (Section 8.1). A dedicated algorithm that exploits the structural mechanism of Section 8.2—directly optimising moves that tighten $G^+(\sigma)$—could improve on the general-purpose tabu search used here.

- Other rankings. Additional admissible orders on intervals [6] could be tested with the same protocol; the structural mechanism uncovered for LEX2 should generalise to any operator that strictly prioritises one extreme.

## 9. Conclusions

I have presented a theoretical framework for neighbourhood-based local search in the Interval Job Shop Scheduling Problem grounded in the concept of *extreme critical paths* — arcs that are critical in either the best-case or worst-case scenario graph. The resulting neighbourhood $H(\sigma)$ satisfies feasibility, connectivity and the no-loss-of-improving-neighbours property for all four admissible interval rankings considered (EV, LEX1, LEX2, YX) simultaneously.

Five neighbourhood variants ($N_1$, $N_2$, $N_3$, $N_{\text{ext}}$, $N_8$) and four ranking operators were evaluated on 82 benchmark instances of up to 1000 operations, across three phases: a $5 \times 4$ comparison at a common parameter setting, a five-way comparison after irace-based tuning, and a comparison with three published IJSP metaheuristics.

After per-neighbourhood irace-tuned tabu search, $N_2$ and $N_8$ emerge as the two strongest variants—statistically tied (negligible effect $r = 0.07$)—while $N_{\text{ext}}$, $N_1$, and $N_3$ trail by medium, large, and very large effects respectively. Considered jointly with runtime, $N_2$ is the most efficient choice across all instance sizes: it ties for best quality, is faster than $N_{\text{ext}}$ for all instances with $\geq 450$ operations, and is faster than $N_8$ for small and medium instances. $N_8$ is the preferred choice for large instances when runtimes of 200–600 s are acceptable, as it achieves the best quality on the tai50×15 class. Regarding ranking operators, LEX2 yields makespan intervals 5–11% narrower than the alternatives on average. A float analysis of both extreme graphs provides structural support for this finding: LEX2 solutions show more critical operations and less per-operation slack in $G^+(\sigma)$, consistent with LEX2 directly

tightening the worst-case critical path rather than leaving more structural slack. The $N_8$ variant's rise to the top tier under tabu search—in contrast to its mid-table performance under hill climbing—is a key finding: extra-block reinsertion moves provide no advantage under first-improvement hill climbing (the cost exceeds the gain), but become genuinely useful under tabu search, where the memory mechanism exploits the enlarged candidate set without cycling. This underscores that the choice of local-search strategy interacts strongly with neighbourhood design in the interval setting. The comparison against three published IJSP metaheuristics (GA, $ABC_{E3}$, ESABC) provides external validation of the approach: the irace-tuned TS-$N_2$ achieves strictly lower average RE (%) than the previous best solver on every one of the 82 benchmark instances, with mean reductions of 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances.

These results provide actionable guidance for practitioners implementing local search for uncertain job shop scheduling: the choice of ranking operator can be used not only to optimise expected makespan but also to directly control solution uncertainty, and the choice of neighbourhood should account for whether the problem is genuinely interval-valued or only nominally uncertain.

## CRediT authorship contribution statement

**Hernán Díaz Rodríguez**: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

## Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

The raw experimental data underlying every figure and table in this article (run logs, aggregated CSVs, irace tuning records, instance files, and analysis scripts) are openly available on Zenodo [33]. Per-instance RE (%) results for all 70 Taillard instances are provided as a separate supplementary document [31]. The classical JSP benchmarks used to derive the interval instances are taken from the OR-Library and from [34].

## Acknowledgements

[TODO: Funding sources, project IDs, computing resources.]

## References

[1] P. J. M. van Laarhoven, E. H. L. Aarts, J. K. Lenstra, Job shop scheduling by simulated annealing, Operations Research 40 (1) (1992) 113–125.

[2] E. Nowicki, C. Smutnicki, A fast taboo search algorithm for the job shop problem, Management Science 42 (6) (1996) 797–813. doi:10.1287/mnsc.42.6.797.

[3] E. Nowicki, C. Smutnicki, An advanced tabu search algorithm for the job shop problem, Journal of Scheduling 8 (2005) 145–159. doi:10.1007/s10951-005-6364-5.

[4] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2009.

[5] D. Lei, Interval job shop scheduling problems, International Journal of Advanced Manufacturing Technology 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.

[6] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, Fuzzy Sets and Systems 220 (2013) 69–77. doi:10.1016/j.fss.2012.07.015.

[7] S. Karmakar, A. K. Bhunia, A comparative study of different order relations of intervals, Reliable Computing 16 (2012) 38–72.

[8] D. Dubois, H. Fargier, P. Fortemps, Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge, European Journal of Operational Research 147 (2003) 231–252.

[9] C. Artigues, C. Briand, T. Garaix, Temporal analysis of projects under interval uncertainty, in: C. Schwindt, J. Zimmermann (Eds.), Handbook on Project Management and Scheduling Vol. 2, Springer, 2015, pp. 911–927. doi:10.1007/978-3-319-05915-0_11.

[10] I. González Rodríguez, C. R. Vela, J. Puente, R. Varela, A new local search for the job shop problem with uncertain durations, in: Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling (ICAPS-2008), AAAI Press, Sydney, Australia, 2008, pp. 124–131.

[11] J. Xie, X. Li, L. Gao, L. Gui, A new neighbourhood structure for job shop scheduling problems, International Journal of Production Research 61 (7) (2022) 2147–2161. doi:10.1080/00207543.2022.2053490.

[12] Z. Xu, R. R. Yager, Some geometric aggregation operators based on intuitionistic fuzzy sets, International Journal of General Systems 35 (4) (2006) 417–433. doi:10.1080/03081070600574353.

[13] I. González Rodríguez, C. R. Vela, J. Puente, A. Hernández-Arauzo, Improved local search for job shop scheduling with uncertain durations, in: Proceedings of the Nineteenth International Conference on Automated Planning and Scheduling (ICAPS 2009), AAAI Press, Thessaloniki, Greece, 2009, pp. 154–161.

[14] J. Puente, C. R. Vela, I. González Rodríguez, Fast local search for fuzzy job shop scheduling, in: H. Coelho, et al. (Eds.), Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010), IOS Press, 2010, pp. 739–744. doi:10.3233/978-1-60750-606-5-739.

[15] C. R. Vela, S. Afsar, J. J. Palacios, I. González-Rodríguez, J. Puente, Evolutionary tabu search for flexible due-date satisfaction in fuzzy job shop scheduling, Computers & Operations Research (2020). doi:10.1016/j.cor.2020.104931.

[16] P. García Gómez, J. J. Palacios, C. R. Vela, I. González-Rodríguez, Adaptive lexicographical optimisation with vague goals in green fuzzy flexible job shop scheduling, Annals of Operations Research (2026). doi:10.1007/s10479-026-07239-1.

[17] S. Afsar, J. Puente, J. J. Palacios, I. González-Rodríguez, C. R. Vela, A hybrid evolutionary approach for lexicographic green flexible job-shop with interval uncertainty, Natural Computing 24 (2025) 483–496. doi:10.1007/s11047-025-10016-x.

[18] H. Díaz, I. Díaz, I. González-Rodríguez, J. J. Palacios, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020), Vol. 1238 of Communications in Computer and Information Science, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3$_5$2.

[19] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Tardiness minimisation for job shop scheduling with interval uncertainty, in: Hybrid Artificial Intelligence Systems (HAIS 2020), Vol. 12344 of Lecture Notes in Artificial Intelligence, Springer, 2020, pp. 209–220. doi:10.1007/978-3-030-61705-9$_1$8.

[20] K. Tamssaouet, S. Dauzère-Pérès, A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems, European Journal of Operational Research 311 (2) (2023) 455–471. doi:10.1016/j.ejor.2023.05.018.

[21] P. García Gómez, S. Dauzère-Pérès, C. R. Vela, I. González-Rodríguez, New bounds for move evaluation in the flexible job-shop scheduling problem, Computers & Industrial Engineering 212 (2026) 111696. doi:10.1016/j.cie.2025.111696.

[22] A. Calamita, C. Meloni, M. Pranzo, M. Samà, Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning, Flexible Services and Manufacturing Journal (2025). doi:10.1007/s10696-025-09640-7.

[23] H. Diaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, Natural Computing 22 (2023) 645–657. doi:10.1007/s11047-023-09953-2.

[24] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, Integrated Computer-Aided Engineering 30 (2023) 223–242. doi:10.3233/ICA-230705.

[25] J. Fortin, P. Zieliński, D. Dubois, H. Fargier, Criticality analysis of activity networks under interval uncertainty, Journal of Scheduling 13 (6) (2010) 609–627. doi:10.1007/s10951-010-0163-3.

[26] D. Dubois, H. Fargier, J. Fortin, Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks, Journal of Intelligent Manufacturing 16 (2005) 407–421.

[27] T. Garaix, C. Artigues, C. Briand, Fast minimum float computation in activity networks under interval uncertainty, Journal of Scheduling (2013). doi:10.1007/s10951-012-0272-2.

[28] J. E. Beasley, OR-Library: distributing test problems by electronic mail, Journal of the Operational Research Society 41 (11) (1990) 1069–1072. doi:10.1057/jors.1990.166.

[29] É. Taillard, Benchmarks for basic scheduling problems, European Journal of Operational Research 64 (2) (1993) 278–285. doi:10.1016/0377-2217(93)90182-M.

[30] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, Operations Research Perspectives 3 (2016) 43–58. doi:10.1016/j.orp.2016.09.002.

[31] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Supplementary material: per-instance RE (%) on Taillard IJSP instances (TA1–TA70) (2026). doi:TODO: 10.5281/zenodo.XXXXXXX.
URL `https://zenodo.org/record/XXXXXXX`

[32] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimisation in job shop with interval uncertainty, Logic Journal of the IGPL (2022). doi:10.1093/jigpal/jzac016.

[33] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Experimental dataset: Neighbourhood structures and ranking operators for the interval JSP (raw runs, configurations, instances, scripts) (2026). doi:TODO: 10.5281/zenodo.XXXXXXX.
URL `https://zenodo.org/record/XXXXXXX`

[34] Optimizizer, Best known solutions for the Taillard job shop instances, `http://optimizizer.com/TA.php`, accessed: May 2026.